

Conceitual - Importância da Preparação

No desenvolvimento de modelos, a preparação dos dados costuma tomar mais tempo que a modelagem em si, e por um bom motivo: a performance final está limitada à qualidade do que você entrega ao algoritmo. Dados com escalas muito diferentes, excesso de ruído ou desbalanceados acabam sabotando até as técnicas mais avançadas. No fim das contas, a regra é clara: se a entrada é ruim, a saída também será.

Valores Ausentes

Lidar com valores ausentes é quase uma arte. Você tem o caminho da exclusão, que é rápido mas perigoso pela perda de informação, e o caminho da imputação, onde preenchemos os vazios com a média, mediana ou até previsões. O ponto central aqui é entender o contexto: às vezes, o fato de um dado estar faltando já é uma informação importante que não deve ser simplesmente apagada.

Normalização vs Padronização

Algoritmos que calculam distâncias se perdem se as variáveis estiverem em escalas muito diferentes, como comparar idade com salário bruto. Para resolver isso, usamos a Normalização para comprimir os valores em um intervalo fixo (geralmente entre 0 e 1) ou a Padronização para centralizar os dados em torno da média, tornando o modelo mais estável e justo na análise das variáveis.

Dados categóricos

O cuidado aqui é não deixar o modelo interpretar categorias como se fossem números sequenciais. Simplesmente atribuir valores como 1, 2 e 3 pode criar uma hierarquia falsa. O ideal é converter essas variáveis usando técnicas como o One-Hot Encoding, criando colunas que o computador entenda sem assumir que uma categoria é "maior" que a outra.

Overfitting e Separação de Dados

Para garantir que o modelo aprenda padrões reais em vez de apenas decorar os exemplos, a separação entre treino, validação e teste é inegociável. O erro mais comum é o "vazamento de dados" (data leakage), que ocorre quando informações do teste acabam influenciando o treino. Isso gera resultados excelentes no papel, mas que falham ao enfrentar dados reais no dia a dia.

Balanceamento de Classes

Quando temos uma classe muito mais frequente que as outras, como em detecção de fraudes, onde quase tudo é transação legítima, o modelo tende a ignorar a minoria. Para equilibrar o jogo, usamos técnicas como SMOTE para criar dados sintéticos da classe menor, undersampling para reduzir a maior, ou ajustes de pesos para dar mais importância aos eventos raros.

Detecção de Outliers

Nem todo dado é bem-vindo. Valores muito fora da curva podem ser erros de coleta ou casos raros valiosos. Usando métodos como IQR ou Z-score, conseguimos isolar esses pontos. A decisão de mantê-los ou removê-los deve ser estratégica: eles podem poluir o aprendizado ou ser justamente a anomalia que você está tentando prever.

Feature Engineering

É aqui que transformamos dados brutos em inteligência. Ao converter uma data de nascimento em idade, ou coordenadas de GPS em um nome de bairro, você "mastiga" a informação para o modelo. Essa etapa simplifica o caminho para o algoritmo, permitindo que ele foque no que realmente importa para a predição.

Pipeline de Pré-processamento

Para que todo o processo seja organizado e replicável, estruturamos um pipeline. Ele é o fluxo que amarra desde a limpeza inicial e tratamento de nulos até o escalonamento e a divisão final dos dados. Isso garante que, sempre que novos dados chegarem, eles passem exatamente pelos mesmos critérios de qualidade antes de chegar ao modelo.

Estudo de Caso Aplicado

Em uma análise de crédito, por exemplo, o rigor na preparação é o que garante a segurança da operação. É preciso corrigir rendas inconsistentes, tratar o estado civil sem criar ordens artificiais, ajustar as escalas de salário e idade e, principalmente, garantir que o modelo não ignore os perfis de risco (geralmente minoritários). Sem esse preparo, o banco corre o risco de aprovar créditos baseados em distorções estatísticas.